

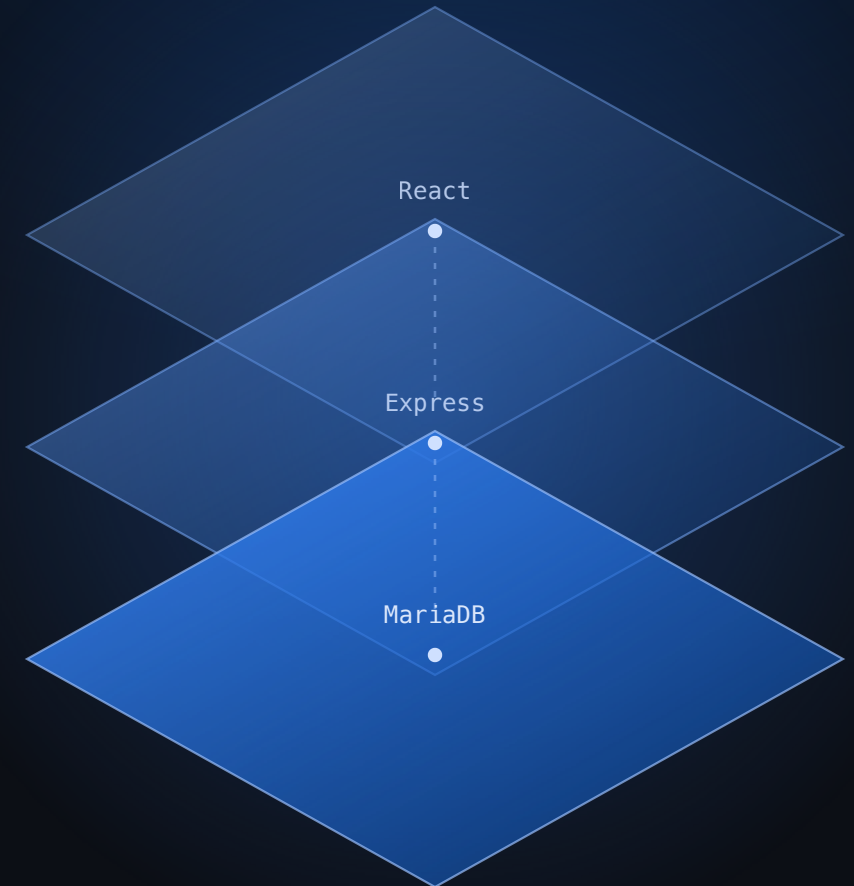
TÜV Prüfstelle Pro

Webanwendung zur digitalen Abbildung eines Prüfstellen-
Workflows

Im Fokus: Wie werden kritische Fachregeln zuverlässig durchgesetzt?

Marwan Saleh · Oussama Hlayhel

Hochschule Hannover · Fakultät II



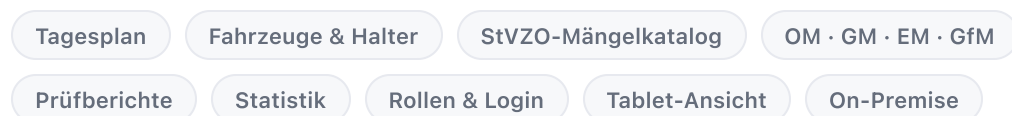
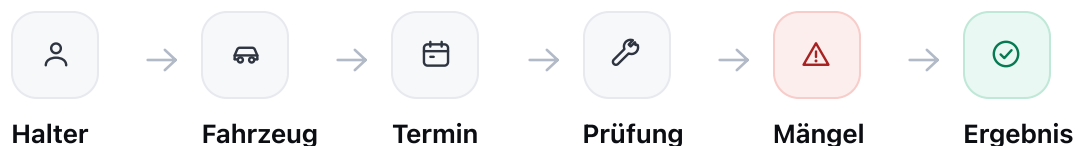
 React

 Express

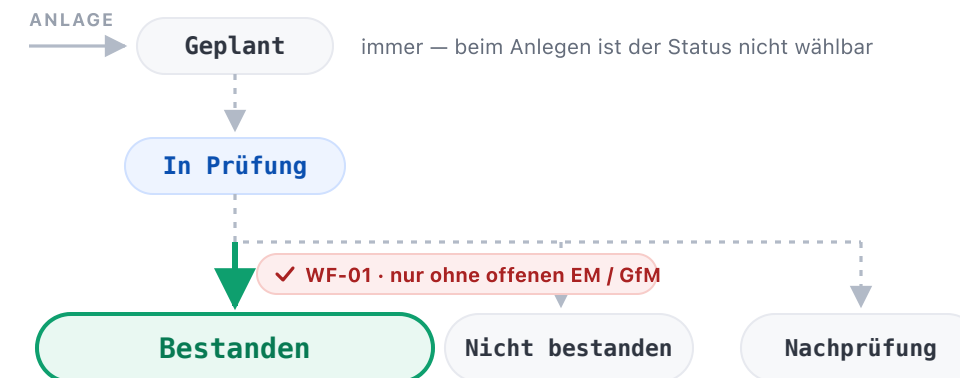
 MariaDB

Ein Ablauf, kein Formular.

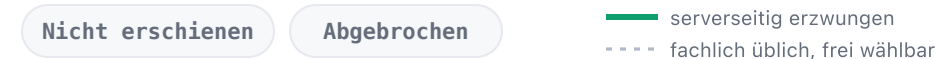
DER PRÜFABLAUF



TERMIN-STATUS — 7 WERTE, EINE GESCHÜTZTE KANTE



WEITERE ENDZUSTÄNDE



„Bestanden“ ist kein Wert, den man einfach setzt — seine Zulässigkeit hängt an den Mängeln des Termins.

WF-01

§ 29 STVZO

Offener **EM** oder **GfM** → kein Bestanden.

Der Prüfer erfasst einen Mangel — und das Ergebnis kippt.

01 PRÜFUNG LÄUFT · KEINE MÄNGEL

The interface shows a header 'Mängelerfassung' with three dots. Below it, a section 'Erfasste Mängel' has a counter '0'. A large grey box contains the text 'Keine Mängel erfasst' and 'Fahrzeug bisher ohne Beanstandungen'. At the bottom, a section 'PRÜFERGEBNIS SETZEN' contains six buttons: 'Bestanden' (highlighted in green), 'Nicht bestanden', 'In Prüfung', 'Nachprüfung', 'Abgebrochen', and 'Nicht erschienen'.

„Bestanden“ ist wählbar.

02 ERHEBLICHER MANGEL (EM) ERFASST

The interface shows the same header. The 'Erfasste Mängel' counter is now '1'. A blue box displays the fault: '1.2.1 Betriebsbremse: Bremsleitung stark korrodiert'. Below it, a red box with a warning icon says 'Hauptmangel vorhanden — nicht bestanden' and '1xEM'. The 'PRÜFERGEBNIS SETZEN' section now has 'Nicht bestanden' highlighted in red, while 'Bestanden' is greyed out.

„Bestanden“ ist gesperrt — die Schaltfläche ist deaktiviert, der Hinweis nennt § 29 StVZO.

03 SONDERFALL · TERMIN WAR BEREITS „BESTANDEN“

The interface shows a header with three dots. Below it, a section displays vehicle information: 'HH-AB 5678 Volkswagen Golf VIII GTI'. To the right, a red box says 'Nicht bestanden' and a grey box says 'NICHT VERKEHRSSICHER'. Below this, a summary line reads: 'Hauptuntersuchung (HU) · A. Fischer · Sabine Koch · 1 Mangel · ~45 min'.

Dann wird ein blockierender Mangel erfasst — und der Server stuft den Termin **selbst zurück**. Niemand hat geklickt. Warum beides zusammengehört: Folie 07.

Das ist, was der Benutzer sieht.
**Kann man diesem „Bestanden“
trauen?**

Wir haben nicht behauptet, dass es hält. Wir haben es angegriffen.

01 📄 Über die Oberfläche

Der reguläre Weg: Prüfer klickt auf „Bestanden“.

Schaltfläche deaktiviert

⊗ UI-Guard — abgewiesen

Gute Bedienung. Aber der Client läuft auf einem fremden Rechner.

02 📄 Am Frontend vorbei

Direkter Aufruf der Status-Route mit curl.

```
$ curl -X PATCH ../api/termine/<id>/status \
-d '{"statusCode":"Bestanden"}'
```

```
HTTP/1.1 200 OK
{ "ok": false,
  "reason": "BESTANDEN nicht möglich bei
            erheblichem oder gefährlichem
            Mangel (§29 StVZO)" }
```

⊗ API-Guard — fachlich abgelehnt

03 📄 An der Anwendung vorbei

Rohes SQL direkt gegen MariaDB — kein Frontend, keine API.

```
$ mariadb -u tuv_app tuv_workflow -e \
"UPDATE termin SET status_code=
'Bestanden' WHERE termin_id='...';"
```

```
ERROR 1644 (45000) at line 1:
WF-01: BESTANDEN nicht moeglich bei
erheblichem oder gefaehrlichem
Mangel (§29 StVZO)
```

✓ DB-Trigger — Status unverändert



Die eigentliche Schwäche lag im Nachweis, nicht im System. Angriff 3 war fest auf `docker exec` verdrahtet — in der CI gab es keinen solchen Container, also wurde er dort übersprungen. Heute wählt `dbCli.js` den Zugriffsweg zur Laufzeit, und ein CI-Schritt schlägt fehl, wenn dieser Test still übersprungen wird.

Drei Schichten — drei Verantwortungen.



Der Browser darf unterstützen. Der Server muss **entscheiden**. Die Datenbank muss **schützen**.

BETRIEB

docker compose up Server-PC in der Prüfstelle

API im LAN · :8787

MariaDB nur 127.0.0.1

STAND

React 19

Vite 8

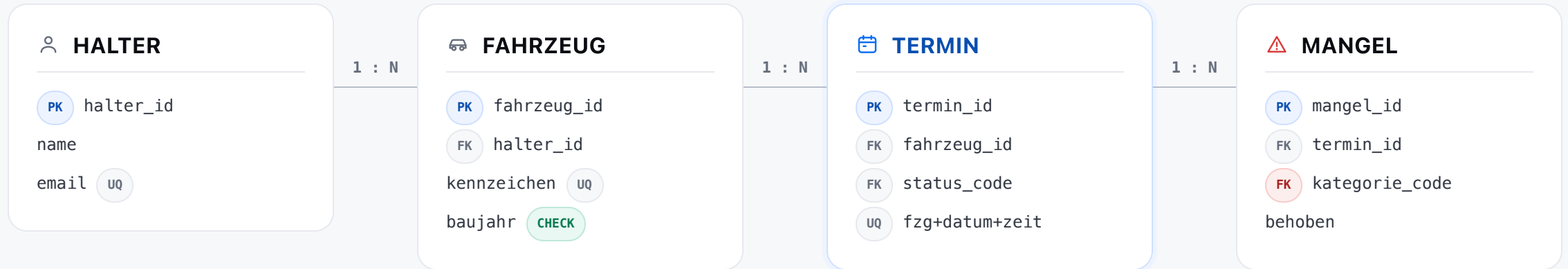
Express 5

MariaDB 11.4

node >= 22

TypeScript schrittweise

Die Fachlichkeit sind die Beziehungen.



10

TABELLEN · 3NF

Löschregeln stehen im Schema, nicht im Code: nach unten CASCADE, ein Halter mit Fahrzeugen ist RESTRICT-geschützt.

Migrationen: versioniert · append-only · Buchführung in schema_migration

🔑 Die Regel lebt als Datum, nicht als hart codierter String

mangel_kategorie.blockiert_bestanden — dieselbe Quelle für UI, API und Trigger. „EM“ steht an keiner Stelle im Code.

Zwei Schreibschritte, ein Ergebnis.

Ein Termin steht bereits auf Beständen — und jetzt wird ein blockierender Mangel erfasst. Der Trigger greift hier nicht: Er hängt am Status-UPDATE, nicht am Mangel-INSERT.

SERVER/INDEX.JS · POST /API/MAENGEL

```
const { mangel, terminDemoted } = await withTransaction(async (conn) => {  
  await conn.query(`INSERT INTO mangel ...`);  
  
  // blockierende Kategorie + Termin steht auf Beständen?  
  if (!istBehoben && bool(kat?.blockiert_bestanden)) {  
    if (t?.status_code === 'Bestanden') {  
      await conn.query(  
        "UPDATE termin SET status_code = 'Nicht bestanden' ...";  
      );  
      demoted = true;  
    }  
  }  
});
```

Ausschnitt aus dem Quelltext — withTransaction klammert beide Schreibschritte.

- 1 **BEGIN**
 - 2 **Mangel speichern**
INSERT · EM, behoben = false
 - 3 **Termin zurückstufen**
UPDATE · → Nicht bestanden
- ✓ **COMMIT**

⊗ Fehler an irgendeiner Stelle
ROLLBACK

Entweder **beide** Änderungen — oder **keine**.

Der Zwischenzustand — Mangel gespeichert, Termin noch grün — wäre genau der Zustand, den die Anwendung verhindern soll. Und niemand würde ihn bemerken.

BEOBACHTET AM 08.09.2026 · ECHTE API-ANTWORT

```
{ "mangel": {  
  ..., "kategorieCode": "EM" },  
  "terminDemoted": true }
```

Drei Rollen. Eine Stelle, die sie durchsetzt.



Empfang

- Halter, Fahrzeuge, Termine anlegen und ändern

-
- ⊗ kein Prüfergebnis
 - ⊗ keine Mängel



Prüfer

- alles vom Empfang
- Mängel erfassen und entfernen
- Prüfergebnis setzen

-
- ⊗ kein Löschen



Chef

- alles vom Prüfer
- löschen
- Administration (/api/admin/*)

🔑 Admin zusätzlich per Token



Die UI blendet aus.

Komfort. Man sieht nur, was man darf.



Die API verweigert.

401 — Wer bist du? · 403 — Was darfst du? geprüft bei jedem Request, pro Route.

Nicht wie viele Tests. Was sie beweisen.

Jede Ebene nimmt mehr von unserem eigenen System aus dem, was wir glauben müssen.

01

 **Unit & Komponenten**
isolierte Logik, UI-Bausteine — API teilweise gemockt

Vitest

02

 **Server-Integration**
Express gegen **echte MariaDB** · WF-01, Rechte, Validierung

Vitest

03

 **End-to-End**
echter Browser → Frontend → API → MariaDB · **nichts gemockt**

Playwright

04

 **SQL-Bypass**
UI weg · API weg · nur die Datenbank antwortet

stärkster Einzelbeweis

DER TEST — NICHT NUR DER FEHLER

```
// server/tests/wf01.test.js — Layer 3
const { exitCode, stderr } = sqlRunner.run(
  `UPDATE termin SET status_code='Bestanden' WHERE termin_id='...';`);
expect(exitCode).not.toBe(0); // SQL abgelehnt
expect(stderr).toMatch(/ERROR 1644.*45000/);
expect(stderr).toMatch(/WF-01/);
expect(after.statusCode).toBe(termin.statusCode); // unverändert
```

ERGEBNIS DER PIPELINE ZUM LETZTEN MASTER-COMMIT

github.com · Actions · CI #134

Lint · Typecheck · Test · Build summary

Vitest Test Report

Summary

- Test Files:  19 passes · 19 total
- Test Results:  254 passes · 254 total
- Other: 2 skips · 2 total

Job summary generated at run-time

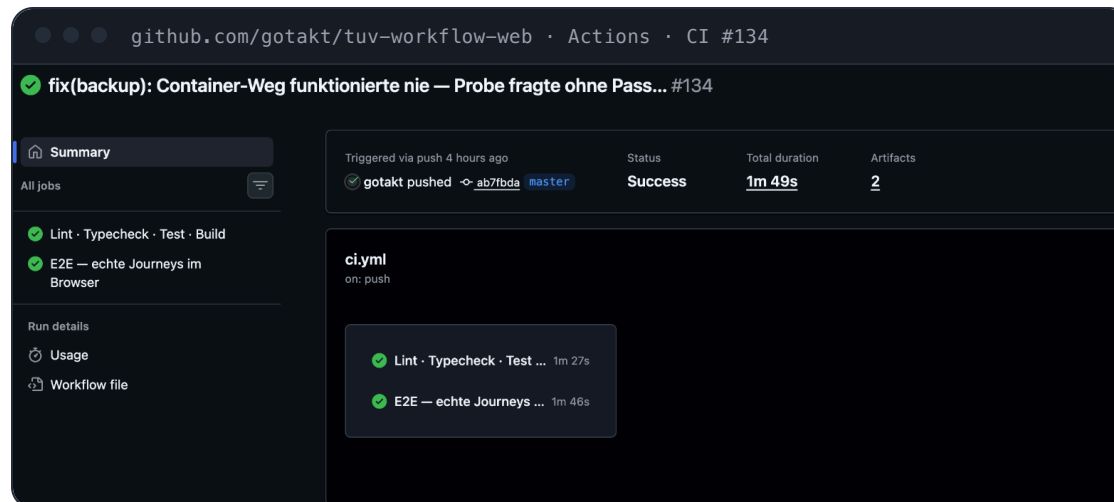
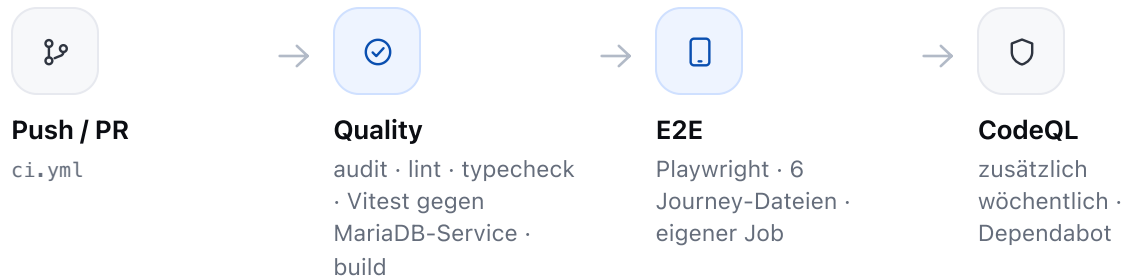
Stand: 08.09.2026 — lokal identisch reproduziert: 254 passed · 2 skipped. Dazu 25 Playwright-Tests in 6 Journey-Dateien, eigener Job.

👁 Ein CI-Schritt schlägt fehl, wenn der SQL-Bypass-Test still übersprungen wird. Ein übersprungener Test sieht in der Zusammenfassung genauso grün aus wie ein bestandener.

Die Datenbankregel ist nicht behauptet.
Sie ist **gezeigt**.

Ein Commit endet nicht beim Push. Er muss die Pipeline überleben.

BEI JEDEM PUSH UND PULL REQUEST AUF MASTER



ZUSÄTZLICH, UNABHÄNGIG VOM COMMIT



UND WENN EIN TAG V* GESETZT WIRD

- Version gegen den Tag prüfen
- audit · lint · typecheck · Tests
- Restore-Drill — inkl. DROP DATABASE
- Paket bauen, auspacken, Prüfsummen verifizieren
- GitHub-Release als draft

👤 Die Automatik baut und prüft. Veröffentlicht wird von Hand.
Der Entwurf bleibt liegen, bis ein Mensch ihn freigibt.

Der Drill zerstört die Datenbank. Wirklich.



...

```
— 3/7 Datenbank zerstören (DROP DATABASE) —  
Datenbank 'tuv_drill' ist weg.  
— 5/7 Fingerabdruck vergleichen —  
vorher / nachher: 298133f0036e2110...f73229 Identisch.  
— 6/7 Geschäftstabellen und WF-01-Trigger —  
termin: 13 ✓ mangel: 13 ✓ benutzer: 3 ✓  
trg_termin_wf01_update ✓  
Trigger greift nach dem Restore (UPDATE abgelehnt) ✓  
— 7/7 Gegenprobe mit beschädigtem Backup —  
Beschädigtes Backup korrekt abgelehnt ✓  
  
RESTORE-DRILL BESTANDEN
```

Ausschnitt aus einem echten Lauf am 08.09.2026 · ./scripts/restore-drill.sh

⚠ Warum die Trigger-Prüfung dazugehört

Ein Dump ohne Trigger spielt sich **ohne sichtbaren Fehler** ein. Die Daten wären vollständig — und WF-01 wäre weg. Deshalb prüft der Restore, ob der Trigger wieder da ist und ob er greift.

Ein Backup ist erst bewiesen, wenn der **Restore getestet** wurde.

🕒 wöchentlich in der CI

bei jeder Skript-Änderung

vor jedem Release

zwei Zugriffswege

Tier 1 · Binlogs aktiv

Tier 2 · Dump umgesetzt

Tier 3 · Offsite: Konzept

React



Express



MariaDB

Kann man diesem „Bestanden“ trauen?

**Ja — weil es nicht davon abhängt,
dass ein Button deaktiviert bleibt.**

🛡️ **Durchsetzen.**

UI · API · Datenbank-Trigger

🧪 **Testen.**

bis hinunter zum rohen SQL

🔄 **Wiederherstellen.**

bewiesen, nicht behauptet

Vielen Dank für Ihre Aufmerksamkeit.

Wir freuen uns auf Ihre Fragen — der Anhang B1–B15 liegt bereit.

WAS WIR HEUTE ANDERS MACHEN WÜRDEN

Die fachliche Ablehnung auch auf HTTP-Ebene klarer modellieren: Der Server antwortet auf ein unzulässiges „Bestanden“ mit 200 und `{ ok: false }` — ein 409 oder 422 würde denselben Fall sichtbarer machen, ohne dass ein Client die Antwort auslesen muss.

Rechte-Matrix — und eine geschlossene Lücke

Die Matrix liegt in server/auth.js (ROLLEN_RECHTE) und wird von der Middleware requireAuth(recht) pro Route erzwungen. Die UI blendet zusätzlich aus.

AKTION	EMPFANG	PRUEFER	CHEF	RECHT
Lesen (alle GET-Endpunkte)	✓	✓	✓	lesen
Halter / Fahrzeuge / Termine anlegen und ändern	✓	✓	✓	schreiben
Status setzen über PATCH /api/termine/:id/status	✗	✓	✓	status
Mängel anlegen und entfernen	✗	✓	✓	status
Löschen · /api/admin/* (zusätzlich X-Admin-Token)	✗	✗	✓	loeschen
Status über den generischen PATCH /api/termine/:id	✗	✗	✗	gar nicht – HTTP 400

Was hier offen war

Der generische PATCH /api/termine/:id nahm statusCode entgegen und verlangte nur schreiben; POST /api/termine übernahm einen mitgelieferten Status. Die Rolle *empfang* konnte damit ein Prüfergebnis setzen — die Matrix galt an dieser Stelle nur auf dem Papier.

Wie es geschlossen wurde

statusCode ist aus dem PATCH-Mapping entfernt und wird mit 400 abgelehnt statt still ignoriert; POST /api/termine legt jeden Termin als Geplant an. Damit bleibt **genau eine** Route für Statusänderungen — die einzige mit requireAuth("status") und WF-01-Guard. Abgesichert durch Regressionstests in termin-status-routen.test.js (ADR-003, Nachtrag).

Fail-closed: Unbekannte Rollen liefern in ROLLEN_RECHTE immer false. Ohne gültigen Token 401, bei unzureichender Rolle 403. **Grenze:** Das Rollenmodell wirkt an der API. Wer direkt in die Datenbank schreibt, umgeht es — dagegen wirken der WF-01-Trigger und die Bindung des DB-Ports an 127.0.0.1.

Physisches Schema — 10 Tabellen

InnoDB / utf8mb4, angelegt durch die versionierten Migrationen in server/migrations.js.

TABELLE	ZWECK	CONSTRAINTS & INDIZES (AUSWAHL)
halter	Fahrzeughalter (eigene Entität, 3NF)	E-Mail UNIQUE · Index auf Name · FK-Ziel für Fahrzeug (RESTRICT)
fahrzeug	Fahrzeug-Stammdaten	Kennzeichen und FIN UNIQUE · CHECK Baujahr 1885–2100, km 0–3 Mio · Indizes hu_faellig, halter_id
termin	Prüftermine	UNIQUE (fahrzeug_id, datum, uhrzeit) · 4 Fremdschlüssel · Index (datum, uhrzeit) · WF-01-Trigger
mangel	Einzelmäängel mit StVZO-Code	FK termin (CASCADE) und kategorie (RESTRICT) · Indizes termin_id, kategorie_code
status	7 Termin-Status mit ist_endzustand	PK status_code · FK-Ziel mit ON UPDATE CASCADE
pruefart	12 Prüffarten (HU, AU, HU+AU, NP, § 21 ...)	PK prueft_code · Seed idempotent per INSERT IGNORE
pruefer	Sachverständige mit Qualifikation	PK pruefer_kuerzel · Termin-FK mit ON DELETE SET NULL
mangel_kategorie	OM / GM / EM / GfM mit blockiert_bestanden	Fachregel als Datum — UI, API und Trigger lesen dasselbe Flag
benutzer	Login-Konten der drei Rollen	kuerzel UNIQUE · CHECK rolle ∈ {empfang, pruefer, chef} · scrypt-Hash · aktiv-Flag
schema_migration	Buchführung der Migrationen	version PK · name · applied_at

Status-Werte: Geplant · In Prüfung · Bestanden · Nicht bestanden · Nachprüfung · Nicht erschienen · Abgebrochen. Endzustand-Flag bei Bestanden, Nicht bestanden, Nicht erschienen, Abgebrochen.

Designprinzip: Endliche Wertemengen sind Fremdschlüsseltabellen statt freier Strings. Dadurch ist WF-01 datengetrieben — der Trigger liest mangel_kategorie.blockiert_bestanden und hat „EM“/„GfM“ nicht hart verdrahtet.

Versionierte Migrationen und Desired-State-Objekte (ADR-011)

Zwei bewusst getrennte Mechanismen: Schemaänderungen laufen genau einmal, Seeds und Trigger bei jedem Start.

Versioniert — genau einmal pro Datenbank

Buchführung in schema_migration mit Version, Name und Zeitstempel. **Append-only:** ausgelieferte Migrationen werden nie editiert — Bestandsinstallationen haben sie schon protokolliert. Änderung = neue Version.

V	NAME	INHALT
1	initial-schema	die 8 Fachtabellen — mit IF NOT EXISTS, damit Bestandsinstallationen sie als No-op durchlaufen
2	hu-richtlinie-kategorien	Alt-Kategorien → OM/GM/EM/GfM; FK ON UPDATE CASCADE benennt die Mangel-Zeilen mit um. Korrigiert auch, dass EM früher nicht blockierte.
3	benutzer-tabelle	Login-Konten der drei Rollen (CHECK auf die Rolle)

GET_LOCK gegen parallele Boots

GET_LOCK('tuv_schema_migration', 30) serialisiert gleichzeitig startende Prozesse.

Desired State — idempotent bei jedem Start

Stammdaten-Seeds (INSERT IGNORE) und der **WF-01-Trigger** (CREATE OR REPLACE TRIGGER) werden bei jedem Serverstart neu deklariert. Konsequenz: Eine Trigger-Korrektur erreicht jede Installation beim nächsten Start — ohne Migration.

Kein Down-Pfad

Rollback läuft über das Backup-Konzept (Binlog + Dump), nicht über ungetestete Reverse-Migrationen. Ein Downgrade auf einen älteren Stand ist nicht vorgesehen (CHANGELOG 1.0.0).

Warum kein Flyway/Liquibase?

MariaDB hat kein transaktionales DDL — der Hauptvorteil schwerer Frameworks existiert dort gar nicht. Was bleibt, ist Buchführung plus Reihenfolge: rund 60 Zeilen. Konsequenz: jede Migration muss wiederholbar sein, falls sie mittendrin abbricht.

Beim Restore relevant: Weil der Trigger Desired State ist, wird er beim nächsten Start ohnehin neu gesetzt — trotzdem prüft restore.sh ihn direkt nach dem Einspielen. Siehe B12.

Anmeldung im Detail — script und HMAC-Token

Bewusst ohne Zusatz-Abhängigkeiten, nur `node:crypto`: Das On-Premise-Deployment soll ohne native Build-Toolchain auskommen, und `server/auth.js` bleibt als reines Funktionsmodul ohne DB-Zugriff unit-testbar.

Passwörter: script, Parameter im Hash

Format `script$N$r$p$salt$hash` ($N = 16384$). Die Parameter stehen im Hash selbst und können später erhöht werden, ohne bestehende Hashes zu invalidieren. Beim Parsen greifen Plausibilitätsgrenzen — ein manipulierter DB-Wert kann script nicht mit absurden Parametern aufrufen.

Token: HMAC-SHA256, 12 Stunden

`base64url(payload).base64url(HMAC-SHA256(payload))` · Payload: Kürzel, Name, Rolle, Ablauf.
JWT-artig, aber kein Standard-JWT — kein Header-Segment, fester Algorithmus, ein `alg=none`-Downgrade ist strukturell ausgeschlossen. Preis: kein Format, das andere Systeme lesen.

Timing-sichere Vergleiche

`timingSafeEqual` bei Passwort-, Token- und Admin-Token-Vergleich; beide Seiten werden vorher gehasht, damit die Puffer gleich lang sind. Unbekannte Kürzel rechnen trotzdem ein script gegen einen Dummy-Hash — die Antwortzeit verrät nicht, welche Konten existieren.

Wann Auth aktiv ist

`AUTH_ENABLED` ohne Wert: **an bei `NODE_ENV=production`**, aus in Entwicklung und CI (dort laufen Requests als Dev-Chef, damit die Integrationstests ohne Login funktionieren). Das Docker-Deployment setzt `NODE_ENV=production`. In den E2E-Tests wird Auth explizit eingeschaltet — sonst wären die Rollen-Journeys nicht prüfbar.

Fail-fast beim Start

Produktion ohne `ADMIN_TOKEN`, oder Auth aktiv ohne Secret → `process.exit(1)` mit klarer FATAL-Meldung. Beide Guards sind mit echten Prozess-Tests abgedeckt. Grundsatz aus den Code-Kommentaren: *lieber laut kaputt als leise unsicher*.

Keine Enumeration, Brute-Force gebremst

Gemeinsame 401-Meldung für „Kürzel unbekannt“ und „Passwort falsch“ · Login-Rate-Limit 10 Versuche / 15 Minuten in Produktion.

Offen

kein Token-Widerruf (zustandslos, 12 h) · kein Passwort-Self-Service · Token im `localStorage` · keine Passwortrichtlinie, keine 2FA.

Dokumentierte Regeln und wo sie erzwungen werden

Aus docs/datenmodell.md § 2.5. Es gibt **genau eine** Regel, die nicht deklarativ ausdrückbar ist — WF-01. Alle anderen liegen als Constraint im Schema.

ID	REGEL	LOGISCHE AUSDRUCKSFORM	ERZWINGUNG
GR-01	Kennzeichen ist eindeutig	UNIQUE(kennzeichen) auf FAHRZEUG	deklarativ
GR-02	FIN ist eindeutig	UNIQUE(fin); mehrere NULL sind in MariaDB erlaubt	deklarativ
GR-03	WF-01 — kein „Bestanden“ bei offenem EM/GfM	relationsübergreifend, nicht als einfacher CHECK ausdrückbar	UI API DB-Trigger
GR-04	Baujahr-Plausibilität	CHECK (baujahr IS NULL OR baujahr BETWEEN 1885 AND 2100)	deklarativ
GR-05	Kilometerstand-Plausibilität	CHECK (kilometerstand IS NULL OR ... BETWEEN 0 AND 3000000)	deklarativ
GR-06	Cascade beim Fahrzeug-Löschen	ON DELETE CASCADE an TERMIN.fahrzeug_id und MANGEL.termin_id	deklarativ

Warum GR-03 anders ist: Ein CHECK darf in MariaDB nur Werte derselben Zeile prüfen. WF-01 ist eine Bedingung über eine andere Tabelle — deshalb der Trigger, deshalb die drei Ebenen. Alle anderen Regeln brauchen keinen Code.

Es gibt keine WF-02 bis WF-05. Im Repository ist genau eine Workflow-Regel als WF-01 benannt (Pflichtenheft F-05, ADR-003). Weitere fachliche Anforderungen sind als F-01...F-11 dokumentiert, aber nicht als Workflow-Regeln in diesem Sinn.

Die drei Ebenen im Quelltext

EBENE 2 · EXPRESS — PATCH /API/TERMINE/:ID/STATUS

```
if (neuerStatus === "Bestanden") {
  const blocker = await one(
    `SELECT COUNT(*) AS count
     FROM mangel m
     JOIN mangel_kategorie mk ON mk.kategorie_code = m.kategorie_code
     WHERE m.termin_id = ?
     AND mk.blockiert_bestanden = TRUE
     AND m.behoben = FALSE`, [req.params.id]);

  if (Number(blocker?.count ?? 0) > 0) {
    res.json({ ok: false, reason: "BESTANDEN nicht möglich ..."});
    return;
  }
}
```

Bewusst HTTP 200: Die fachliche Ablehnung ist kein Protokollfehler, das Frontend bekommt einen einheitlichen Vertrag { ok, reason }. **Kritikpunkt, den wir selbst nennen:** 409 oder 422 wären eine legitime, in REST üblichere Modellierung.

Ebene 1 · React

Die Schaltfläche „Bestanden“ ist disabled, wenn ein unbehobener blockierender Mangel am Termin hängt; der Titel nennt § 29 StVZO. Der E2E-Test prüft genau diese beiden Eigenschaften.

Die bewusst benannte Grenze

Der Trigger hängt an BEFORE UPDATE ON termin. Wird per direktem SQL ein blockierender Mangel zu einem *bereits bestanden* Termin eingefügt, greift er nicht — diesen Fall behandelt POST /api/maengel transaktional.

Sicherheitsnetz im Fehler-Handler

Löst der Trigger doch aus (z. B. nebenläufig eingefügter Mangel zwischen Blocker-Prüfung und UPDATE), mappt der zentrale Handler SQLSTATE 45000 auf HTTP 422.

EBENE 3 · MARIADB — SERVER/DB.JS, DESIRED STATE

```
CREATE OR REPLACE TRIGGER trg_termin_wf01_update
BEFORE UPDATE ON termin FOR EACH ROW
BEGIN
  IF NEW.status_code = 'Bestanden'
    AND OLD.status_code <> 'Bestanden' THEN
    IF EXISTS (
      SELECT 1 FROM mangel m
      JOIN mangel_kategorie mk ON ...
      WHERE m.termin_id = NEW.termin_id
      AND mk.blockiert_bestanden = TRUE
      AND m.behoben = FALSE) THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'WF-01: BESTANDEN nicht moeglich ...';
    END IF;
  END IF;
END
```

Was auf welcher Ebene geprüft wird

Zahlen aus dem CI-Lauf zum letzten master-Commit und einem lokalen Lauf gegen den Docker-Stack, beide vom **08.09.2026**: 254 passed · 2 skipped · 19 Dateien (Vitest) und 25 Playwright-Tests in 6 Journey-Dateien, alle grün.

EBENE / DATEI	UMGEBUNG	WAS SIE BEWEIST
src/tests/utils	keine	Validatoren, Datums- und Mangel-Logik, Grenzwerte
src/tests/components hooks	jsdom	UI-Bausteine, useDb, Auth-Kontext
src/tests/flows	jsdom, API gemockt	Klickpfade — bewusst <i>kein</i> E2E-Ersatz
server/tests/validate	Node	serverseitige Eingabevalidierung inkl. UUID-Format
server/tests/auth	Node	scrypt, HMAC-Token, Rollen-Matrix (fail-closed)
server/tests/admin-token-boot	echter Prozess-Spawn	Start verweigert ohne ADMIN_TOKEN / Auth-Secret
server/tests/wf01	API + echte MariaDB	WF-01 Ebene 2 <i>und</i> Ebene 3, Auto-Demotion, behoben-Semantik
server/tests/termin-status-routen	API + echte MariaDB	Rechte-Regression: Status nur über eine Route
e2e/*.spec.js	Browser + API + MariaDB	die ganze Kette, nichts gemockt

Testentwurfsverfahren

Äquivalenzklassen und Grenzwertanalyse bei den Validatoren; Entscheidungstabellen für die Statuswechsel-Regeln; gezielte Sicherheitsfälle bei Auth (manipulierte Hashes, gefälschte und abgelaufene Tokens, fail-closed bei unbekannter Rolle).

Zwei Vitest-Projekte

frontend (jsdom) läuft parallel; server (node) mit fileParallelism: false — beide Integrationsdateien reseeden dieselbe Datenbank und würden sich sonst die Testdaten löschen.

Die 2 übersprungenen Tests

Platzhalter in den Skip-Zweigen der beiden Integrationsdateien. Sie greifen nur, wenn *kein* Stack läuft, und melden dann verständlich, wie man ihn startet. Läuft der Stack, sind sie die einzigen Skips.

Lücken, die wir ausweisen

CRUD-Integrationstests für /api/halter, /api/fahrzeuge und /api/maengel fehlen. Coverage-Prozente weisen wir bewusst nicht aus — die veröffentlichten stammen von einem Stand vor dem Umbau.

Der Test, der die eigene Anwendung aus dem Weg nimmt

Und die Geschichte, warum er lange existierte, ohne etwas zu beweisen.

SERVER/TESTS/DBCLI.JS — ZUGRIFFSWEG ZUR LAUFZEIT

```
/* Reihenfolge:
 * 1. Direkter MariaDB/MySQL-Client über TCP
 *    (CI, lokales Setup ohne Docker)
 * 2. docker exec <container> mariadb ...
 *    (Compose-Stack)
 * 3. null – Aufrufer skippt mit klarer Begründung
 */
```

Vorher: ein Layer ohne Nachweis

Der Aufruf war fest auf `docker exec tuv-mariadb` verdrahtet. In der CI läuft MariaDB als Service-Container, nicht über Compose — also gab es diesen Container nicht, und der Test wurde per `TUV_SKIP_SQL_BYPASS` übersprungen. Ein Defense-Layer, der in der Pipeline nie geprüft wird, ist kein geprüfter Layer.

Heute

Der Zugriffsweg wird zur Laufzeit gewählt. Übersprungen wird nur, wenn es gar keinen Weg gibt — und selbst das würde die Pipeline auffliegen lassen.

DER CI-SCHRITT, DER EINEN STILLEN SKIP UNMÖGLICH MACHT

```
- name: Assert Layer-3-Test lief (kein stiller Skip)
run: |
  npx vitest run server/tests/wf01.test.js \
    --reporter=verbose 2>&1 | tee wf01.log
  if ! grep -q "\\.*Layer 3 (DB-Trigger)" wf01.log; then
    echo "Layer-3-Test wurde uebersprungen –
        SQL-Bypass nicht geprueft." >&2
    exit 1
  fi
```

```
$ npx vitest run server/tests/wf01.test.js --reporter=verbose
✓ WF-01 Defense-in-Depth > Layer 3 (DB-Trigger): direkter
  SQL-UPDATE wird abgelehnt 45ms
✓ WF-01 Defense-in-Depth > Layer 2 (API-Guard): ok:false 20ms
✓ Happy Path · behoben-Semantik · Auto-Demotion (2)
Tests 6 passed | 1 skipped (7)
```

Eigener Lauf am 08.09.2026 gegen den lokalen Docker-Stack.

Was atomar ist — und was ausdrücklich nicht

Transaktional (`withTransaction`)

- `POST /api/maengel` — INSERT Mangel + ggf. Rückstufung des Termins
- `POST /api/admin/demo` — Seed der Beispieldaten
- `POST /api/admin/reset` — vier DELETES in FK-sicherer Reihenfolge

Ausdrücklich nicht transaktional

- Frontend-Abläufe aus mehreren HTTP-Requests (z. B. „Halter anlegen, dann Fahrzeug anlegen“)
- Es gibt keinen zusammengesetzten Endpunkt dafür — wäre der Kandidat, falls es fachlich wichtig würde

Isolation

InnoDB-Standard `REPEATABLE READ`, nicht bewusst geändert. Für die Größenordnung einer Prüfstelle (5–20 gleichzeitige Nutzer) war das kein Engpass.

Verbindungen kommen aus einem Pool; ohne `COMMIT` rollt MariaDB beim Abbruch zurück. Der Code macht zusätzlich im `catch` ein explizites `ROLLBACK` und gibt die Verbindung im `finally` frei.

Mehrbenutzer-Verhalten im Frontend

Polling alle 5 Sekunden (`src/hooks/useDb.ts`), pausiert bei Hintergrund-Tab und während eines eigenen Schreibvorgangs (`writeInFlight`-Zähler) — sonst könnte ein Poll-Tick mit altem Stand den laufenden Schreibvorgang überschreiben. Optimistische Updates merken sich den Vorzustand und rollen bei Ablehnung zurück.

Ehrliche Grenze — keine Konflikterkennung

Es gibt heute keine Zeilenversion und kein ETag. Bearbeiten zwei Clients dasselbe gewöhnliche Feld, gewinnt effektiv der letzte Schreibvorgang. Wir behaupten **nicht**, dass die Mehrbenutzer-Bearbeitung konfliktfrei ist. **Was trotzdem geschützt ist:** WF-01 hängt nicht am Client-Zustand — API und Trigger prüfen beim Statuswechsel den aktuellen Datenbankstand.

Härtung — und die bekannten Grenzen aus SECURITY.md

helmet + CSP

Security-Header ab Werk. Die CSP verbietet Inline-Scripts; upgrade-insecure-requests ist entfernt, weil das LAN-Modell bewusst über HTTP läuft.

Rate-Limits (nur Produktion)

/api 600/min · /api/admin 30 / 15 min · /api/auth/login 10 / 15 min. In Dev/CI aus, damit Integrationstests nicht anschlagen.

CORS auf das LAN begrenzt

Statt Wildcard: localhost, 127.0.0.1 und private RFC-1918-Netze plus optionale Whitelist über ALLOWED_ORIGINS.

Fehler-Semantik ohne SQL-Leak

Duplikat → **409** · fehlende Referenz → **400** · CHECK-Verstoß → **400** · Trigger-SIGNAL → **422**. 500er geben keine err.message an den Client.

Kleine Angriffsfläche

DB-Port nur 127.0.0.1 · Adminer nur über ein Compose-Profil · Admin-Endpunkte doppelt geschützt (Chef-Rolle *und* X-Admin-Token) · express.json auf 1 MB · durchgängig parametrisierte Queries.

Health mit Substanz

GET /api/health prüft die Datenbank per SELECT 1 — sonst meldet der Endpunkt „ok“, während MariaDB längst weg ist.

Bekannte Grenzen — SECURITY.md § 9, unverändert übernommen

- **Kein HTTPS im LAN.** Anmeldedaten sind für Mitlesende im selben Netz sichtbar.
- **Kein physischer Schutz.** Wer Zugang zum Server-PC hat, hat Zugang zur Datenbank.
- **Keine Token-Rücknahme.** Ein Token gilt bis zu 12 Stunden, auch wenn das Konto gesperrt wird.
- **Kein Änderungsprotokoll.** Es ist nicht nachvollziehbar, wer ein Prüfergebnis gesetzt hat.
- **Keine Passwortrichtlinie, keine 2FA.**
- **Ein gemeinsamer Admin-Token** statt persönlicher Administratorkonten.
- **Kein Schutz gegen einen böswilligen Prüfer.** Wer die Rolle hat, darf Mängel entfernen und danach „Bestanden“ setzen. Dagegen hilft nur ein Audit-Log, keine technische Sperre.

On-Premise-Deployment

Jede Prüfstelle betreibt docker-compose.yml auf einem Server-PC im lokalen Netz.

services:

```
db:      image: mariadb:11.4   container_name: tuv-mariadb
# Port nur an localhost: die API erreicht MariaDB über
# das interne Docker-Netz, Clients im LAN nie direkt.
ports: [ "127.0.0.1:3306:3306" ]
volumes: mariadb_data · my.cnf (Binlog) · ./backups

api:     image: node:24-alpine
# Docker-Deployment = Kunden-Deployment → production.
# Ohne NODE_ENV lief der Container im dev-Modus und der
# Fail-fast-Guard griff nie – /api/admin/* wäre still
# öffentlich gewesen.
NODE_ENV: production   API_PORT: 8787
volumes: ./dist (gebautes Frontend wird mit ausgeliefert)
```

Netzwerkbild

- Arbeitsplätze und Tablets → `http://<server-ip>:8787`
- Express liefert API *und* gebautes Frontend aus `dist/`
- Port 5173 ist der Vite-Entwicklungsserver und läuft beim Kunden nicht
- MariaDB ist im WLAN nicht sichtbar

Laufzeit

Node 24 (Active LTS) im Image, in allen Workflows und in der Dokumentation. `engines.node >= 22`.
Node 20 ist seit dem 30.04.2026 End-of-Life und bekommt keine Sicherheitsfixes mehr.

Optional: Tauri

Eine Desktop-Hülle (`src-tauri/`) um dasselbe gebaute Frontend, falls ein Arbeitsplatz lieber ein Fenster als einen Browser-Tab hätte. Der ausgelieferte Betriebsweg ist Docker Compose.

Trade-off: Die Prüfstelle verantwortet Backups, Updates und LAN-Sicherheit selbst — dafür laufen die operativen Daten nicht über einen Auftragsverarbeiter. Sobald ein Offsite-Backup aktiv wird, kommt dieser Punkt zurück auf den Tisch.

Was genau geschrieben, geprüft und wiederhergestellt wird

scripts/backup.sh — Tier 2

- voller Dump **mit Triggern**, AES-256-CBC, PBKDF2 mit 200 000 Runden
- Rotation 7 täglich / 4 wöchentlich / 12 monatlich
- je Sicherung eine .meta-Datei mit klartext_sha256
- ohne Schlüssel bricht das Skript ab, statt Klartext mit Halterdaten zu schreiben

scripts/restore.sh — Ein-Befehl-Restore

- entschlüsseln, dann **Prüfsumme vor dem Überschreiben** gegen die .meta vergleichen
- Ziel-Datenbank neu anlegen, einspielen
- danach: Tabellenzahl und **Existenz des WF-01-Triggers** prüfen — fehlt er, bricht das Skript mit Fehler ab
- Rückfrage vor dem Überschreiben einer Produktionsdatenbank

Der Grund für die Trigger-Prüfung: Ein Dump ohne `--triggers` spielt sich ohne sichtbaren Fehler ein. Die Daten wären vollständig, WF-01 wäre weg — und niemand würde es merken, bis jemand ein falsches „Bestanden“ setzt.

Der Drill in sieben Schritten

- Fingerabdruck der Quell-DB (normalisierter Dump, sha256)
- verschlüsseltes Backup schreiben
- **DROP DATABASE** — wirklich, nicht simuliert
- Restore über `restore.sh`
- Fingerabdruck vergleichen — muss identisch sein
- Zeilenzahlen aller Tabellen; Trigger existiert **und greift** (gesperrtes UPDATE muss scheitern)
- Gegenprobe: beschädigtes Backup **muss** abgelehnt werden

Schutz gegen Unfälle

Der Drill läuft nur gegen eine Datenbank, deren Name mit `tuv_drill` beginnt — oder mit explizit gesetztem `TUV_DRILL_ERLAUBE_ZIEL_DB`. Sonst könnte ein Tippfehler die Produktionsdatenbank löschen.

Zwei Zugriffswege, aus gutem Grund

Ein CI-Job mit installiertem MariaDB-Client, ein zweiter erzwungen über `TUV_DB_MODUS=docker`. Genau der zweite deckte auf, dass die Verbindungsprobe des Container-Wegs ohne Passwort anfragte und deshalb nie funktionierte — auf einem Kunden-Server-PC ohne Client hätte das bedeutet: kein Backup, kein Restore.

Offen (docs/backup.md § 7)

Zeitsteuerung am Server-PC (cron / Aufgabenplanung — gehört ins Kunden-Setup) · `sync-offsite.sh` für Tier 3 · Alarmierung bei Fehlschlag am Standort · Point-in-Time-Recovery über Binlogs ist konfiguriert, aber Handarbeit und nicht automatisiert geprüft.

Playwright — 6 Journey-Dateien · 25 Tests

```
// eigene Ports statt der Entwicklungs-Ports
API_PORT = 8788   WEB_PORT = 5174   DB = tuv_e2e

AUTH_ENABLED = true
// im Dev-Default waere sie aus – ohne Login liessen sich
// die Rollen-Journeys gar nicht pruefen

fullyParallel: false   workers: 1
// die Specs teilen eine DB und setzen sie pro Datei neu auf

projects: chromium (Desktop) · tablet (820×1180, hasTouch)
retries: CI ? 1 : 0   ·   trace/video: retain-on-failure
```

Warum eigene Ports?

Auf einer Entwickler-Maschine läuft auf 8787 oder 5173 oft schon etwas anderes. Ein E2E-Lauf, der versehentlich gegen einen fremden Dienst testet, ist schlimmer als einer, der gar nicht startet.

Warum Chromium statt iPad-Profil?

Ein iPad-Profil würde WebKit laden — ein zweiter Browser-Download in jeder Pipeline, und die Aussage wäre trotzdem nur „Safari-Engine“. Geprüft wird das Layout bei 820 px mit Touch.

JOURNEY-DATEI	DECKT AB · STAND 08.09.2026: 25 TESTS GESAMT
01-anmeldung	falsche Zugangsdaten, unbekanntes Kürzel, Anmeldung je Rolle, Abmelden
02-pruefablauf	offener Hauptmangel sperrt „Bestanden“ · neuer Mangel sperrt ein bereits bestandenes Ergebnis · Entfernen gibt frei · Server lehnt auch bei umgangener Oberfläche ab
03-rollen	Empfang ohne Ergebnis-Schaltflächen · Server lehnt Statuswechsel des Empfangs ohne UI ab · Löschversuch des Prüfers · ohne Token 401
04-stammdaten	Pflichtfelder, ungültiger Kreis-Code, Fahrzeug + Termin anlegen, Status beim Anlegen gesperrt auf „Geplant“
05-berichte-statistik	Berichtsliste und Vorschau, leerer Suchzustand, Kennzahlen
06-mobil	Tablet-Viewport: Anmeldung und Navigation, kein horizontales Scrollen

Die Fehlerklasse, die nur hier auffällt: ein Vertragsbruch zwischen Frontend und API — falscher Feldname, geänderte Antwortform. Die Flow-Tests glauben ihrem Mock, die Server-Tests sehen den Browser nicht.

Im Quelltext dokumentiert: Mit absichtlich ausgebautem Server-Guard blieben die reinen Klick-Journeys grün, weil die Oberfläche die gesperrte Aktion nie abschickt. Deshalb gibt es die Tests, die an der Oberfläche vorbei senden.

Vier Workflow-Dateien, ein Paket

DATEI	AUSLÖSER	INHALT
ci.yml	push · pull_request auf master	Job 1: audit · lint · typecheck · Vitest gegen MariaDB-Service · Assert Layer-3 · build. Job 2: Playwright-E2E mit eigenem MariaDB-Service.
codeql.yml	push · pull_request · schedule (Mo 03:23 UTC)	statische Sicherheitsanalyse
restore-drill.yml	push/PR auf scripts/**, migrations.js, db.js · schedule (Mo 04:17 UTC) · workflow_dispatch	zwei Jobs: Drill über den Client-Weg und über docker exec
release.yml	Tag v* · workflow_dispatch	Versionsprüfung · Tests · Restore-Drill · Paket bauen und verifizieren · Release als draft

Kein eigener E2E-Workflow: Die Playwright-Tests sind ein zweiter Job innerhalb von ci.yml, kein separater Workflow. Dazu kommt dependabot.yml für npm-Pakete und GitHub-Actions.

INHALT VON TUV-PRUEFSTELLE-PRO-V1.0.0.TAR.GZ

enthalten

- docker-compose.yml, docker/mariadb/my.cnf, .env.example
- package.json + package-lock.json
- server/ und dist/ (gebautes Frontend)
- scripts/backup.sh, restore.sh, lib/db.sh
- INSTALLATION.md, SECURITY.md, docs/backup.md, docs/mariadb-setup.md
- VERSION und PRUEFSUMMEN.txt (sha256 je Datei)

bewusst nicht

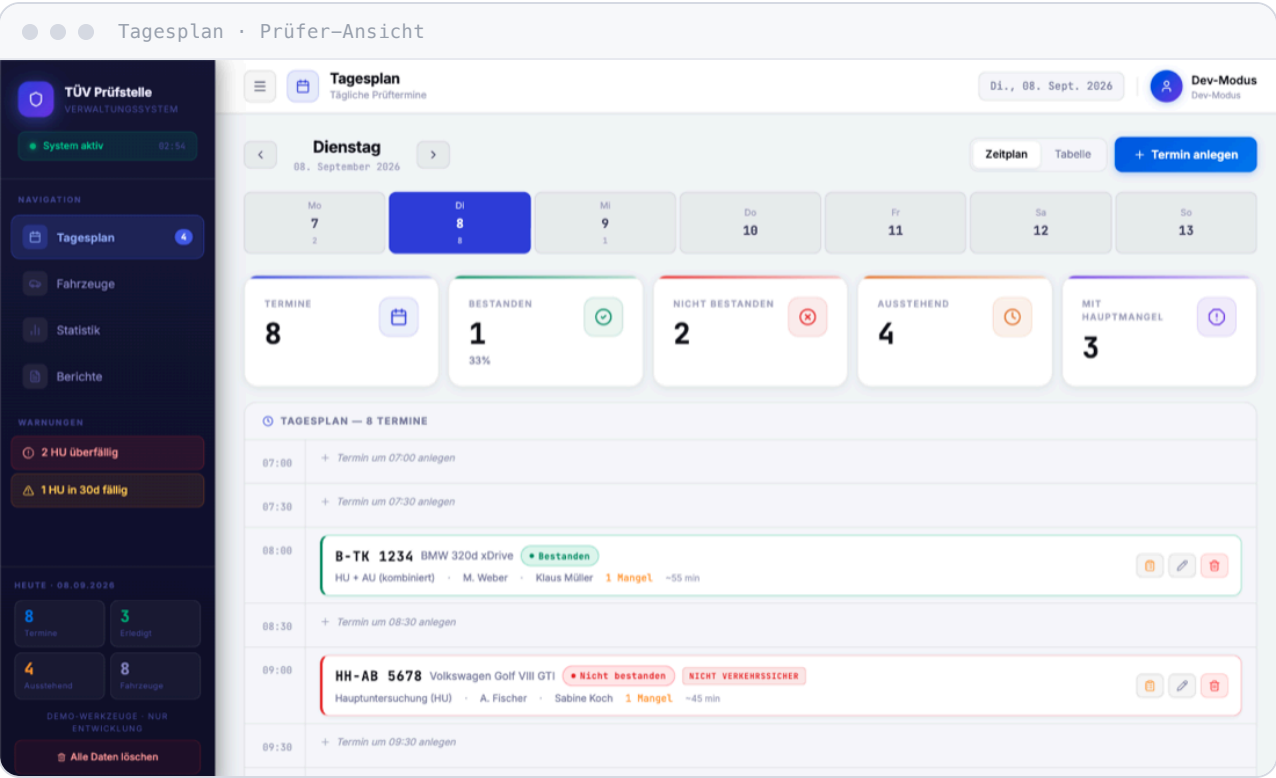
- Frontend-Quellcode
- Tests und Testdaten
- ADRs, Projektunterlagen, Präsentation

Wer ein Paket auspackt, soll sehen, was er betreibt — und nicht erst sortieren müssen.

Aktualisieren einer Bestandsinstallation

./scripts/backup.sh → docker compose down → Paket auspacken, bestehende .env übernehmen → docker compose up -d. Die Migrationen laufen beim Start selbst und sind append-only.

Die laufende Anwendung — und der Demo-Ablauf



Echter Screenshot vom 08.09.2026, Demo-Daten aus POST /api/admin/demo. Der 09:00-Termin ist der aus Folie 03 — „Nicht bestanden“ nach der automatischen Rückstufung.

DEMO-ABLAUF — UNTER ZWEI MINUTEN

- # SCHRITT
- 1

Als **Prüfer** anmelden (Kürzel MW)
- 2

Termin im Tagesplan öffnen · „Mängel erfassen“
- 3

Einen **erheblichen Mangel (EM)** erfassen
- 4

Zeigen: „Bestanden“ ist deaktiviert, Hinweis nennt § 29 StVZO
- 5

Erklären: Server *und* Datenbank schützen den Übergang unabhängig davon
- 6

Den Test-Mangel entfernen · „Bestanden“ ist wieder wählbar

Nicht demonstrieren: einen bestehenden Mangel nachträglich auf behoben = true setzen — dafür gibt es in UI und API aktuell keinen Weg. Mängel lassen sich anlegen und entfernen.

Vorher vorbereiten: Läuft der Server mit ADMIN_TOKEN (Produktionsmodus, nötig für den Login), blendet die Sidebar „Beispieldaten laden“ aus — die Admin-Endpunkte verlangen einen Header, den der Browser bewusst nicht kennt. Demo-Daten also **vor** der Präsentation einspielen, nicht live.